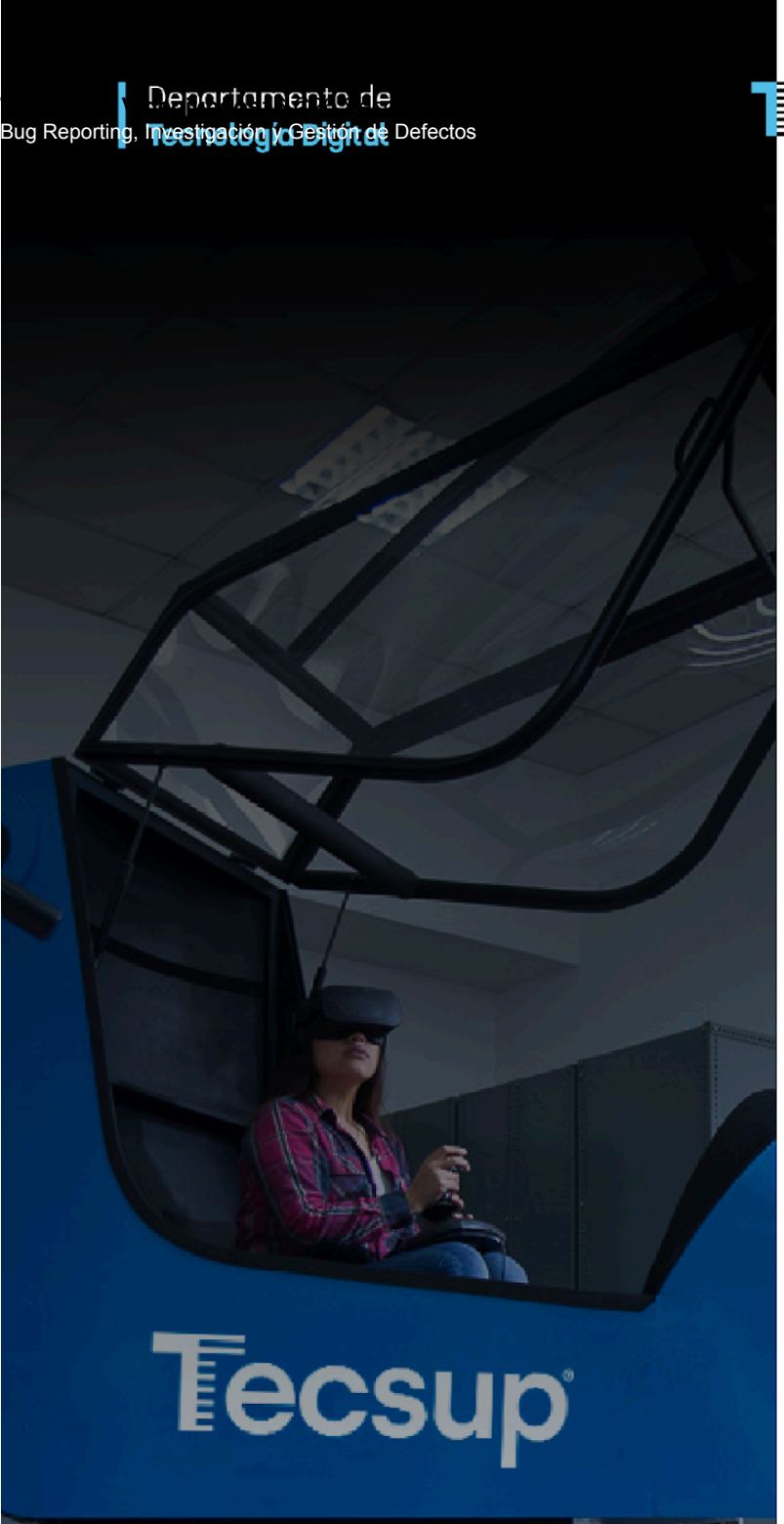
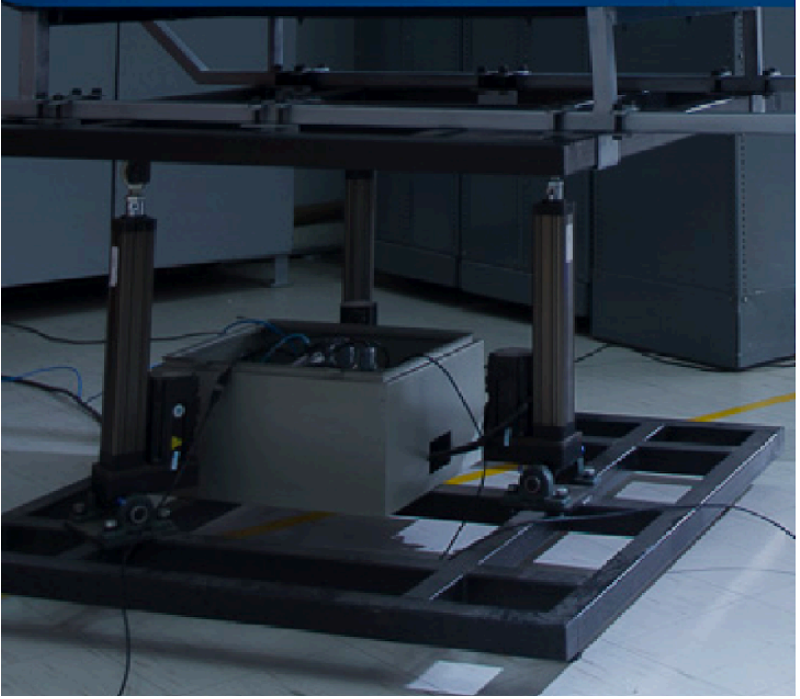
Tecsup TECNOLOGÍA CON SENTIDO





LABORATORIO N° 04

Diseño y Desarrollo de Simuladores y Videojuegos

DOCENTE:

Bestard Aroche, Yunior

CURSO:

Testing de Videojuegos
6 - C26 - Sección A

TESTING DE VIDEOJUEGOS: Bug Reporting, Investigación y Gestión de Defectos

I. Capacidades

- Analizar las funcionalidades implementadas en el proyecto de tesis a partir de las actividades registradas en Trello.
- Diseñar una estrategia de testing orientada a los riesgos y características actuales del videojuego o simulador.
- Ejecutar pruebas sobre una build identificable del proyecto y detectar defectos reproducibles.
- Elaborar Bug Reports profesionales utilizando precondiciones, pasos mínimos, Expected Result, Actual Result, Repro Rate, severidad, prioridad y evidencia.
- Realizar un Bug Triage básico para determinar qué defectos deben resolverse primero e incorporarlos nuevamente a la planificación del proyecto.

II. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red, así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en el lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizadas limpias y ordenadas.

III. Fundamento teórico

Antes de iniciar el laboratorio, revise el material de la sesión “Bug Reporting, Investigación y Gestión de Defectos”. Durante la práctica se aplicarán los conceptos de Build/Environment, Preconditions, Expected Result, Actual Result, Repro Rate, Minimal Reproduction Steps, Evidence, Severity, Priority, Bug Lifecycle, Duplicate, Cannot Reproduce, Reopened y Bug Triage.

IV. Normas empleadas

- No aplica.

V. Recursos

- Proyecto de tesis del equipo y una build funcional del videojuego o simulador.
- Tablero Trello del proyecto con las actividades planificadas, en desarrollo y finalizadas.
- Computadora del laboratorio con Windows 11 y herramientas utilizadas habitualmente por el equipo.
- Software para captura de pantalla o grabación de video; logs o registros del videojuego cuando correspondan.

VI. Metodología para el desarrollo de la tarea

El laboratorio se desarrollará en los equipos de trabajo del proyecto de tesis. Cada equipo evaluará su propio producto y realizará el flujo completo: Trello → selección del alcance → estrategia de testing → ejecución → detección e investigación de bugs → documentación → triage → actualización del proyecto.

> **Nota:** *No se reportarán como bugs funcionalidades que todavía estén pendientes de implementación en Trello. El alcance debe basarse en características que el equipo considere implementadas y listas para ser probadas.*

VII. Procedimiento

FASE 1 — Revisar el estado actual del proyecto

Antes de ejecutar el videojuego, ingresar al tablero de Trello del proyecto de tesis. Revisar las actividades planificadas, en desarrollo y finalizadas. Seleccionar como mínimo cinco actividades o funcionalidades finalizadas que puedan ser evaluadas durante la sesión.

Registrar la selección en la siguiente tabla y adjuntar una captura del tablero de Trello donde se observen las tareas seleccionadas.

N.°	Actividad / funcionalidad finalizada	Estado en Trello	Comportamiento esperado	Riesgo

1	Configurar assemblies para testeo	Finalizada	Crear Minigame.asmdef (refs: InputSystem, TMP, UnityEngine.UI) y EditModeTests.asmdef con includePlatforms: ["Editor"] para que el Test Runner detecte las pruebas EditMode.	Alto
2	Implementar suite de 14 pruebas unitarias EditMode	Finalizada	FirstUnitTest, EqualityAssertionTest, BooleanAssertionTest, ExceptionAssertionTest; reglas de estado y casos límite en CoreGameplayTests; progresión de niveles en LevelProgressionTest; comprobación de resolubilidad de niveles en LevelSolvabilityTest	Alto
3	Refactor de testabilidad en VerificationManager	Finalizada	Exponer CurrentLevelIndex, LevelCount, ConfigureLevels(LevelConfig[]) y NextLevel() público para inyección de dependencias.	Alto
4	Fix BUG-002 (FeedbackUI destruida)	Finalizada	Hacer FeedbackUI.ShowMessage() seguro ante instancias destruidas (if (this == null) return;).	Alto
5	Limpieza de suite	Finalizada	Eliminar TestTry.cs (plantilla sin aserciones) y recortar duplicados en CoreGameplayTests.	Alto

FASE 2 — Definir el alcance y diseñar la estrategia de testing

El equipo no debe comenzar simplemente a jugar. Primero deberá definir qué partes del producto serán evaluadas y qué riesgos desea investigar. Seleccionar como mínimo tres áreas relevantes para el proyecto: gameplay, movimiento, colisiones, combate, IA, UI/HUD, controles, audio, VFX, inventario, Save/Load, rendimiento, estabilidad, navegación entre escenas, persistencia u otras propias del producto.

Proyecto: MinigameTry1 - Prótegeté (minijuego de verificación de puertas)

Equipo: Bestard Aroche, Yunior

Build / versión: Editor Unity 6000.3.9f1 (entorno de editor; sin build de player)

Fecha: 30/09/2026

Plataforma / sistema operativo: Windows 11 (Unity Editor)

Dispositivo de entrada: Teclado y ratón (WASD + mouse; E interactuar, F escanear)

Complete la estrategia antes de comenzar las pruebas:

Elemento	Definición del equipo
Funcionalidades que serán probadas	Escaneo de puertas (ScannerSystem), verificación e instanciado de puertas (VerificationManager / DoorSpawner / DoorOption), progresión de niveles, sistema de integridad (IntegritySystem), temporizador (TimerSystem), límite de intentos (StrikeManager), consola de inicio de verificación (VerificationConsole) y retroalimentación al jugador (FeedbackUI).
Áreas de testing seleccionadas	Gameplay / lógica de niveles · Controles e interacción del jugador (PlayerInteractor) · UI / HUD (FeedbackUI) · Estabilidad (excepciones y referencias nulas en tiempo de ejecución).
Riesgos principales identificados	1) Acceso a singletons estáticos (ScannerSystem.Instance, FeedbackUI.Instance) sin validación de nulidad, con riesgo de crash. 2) Configuración de niveles que puede generar puertas correctas fuera del rango de spawn points, volviendo un nivel imposible de completar. 3) Reglas de negocio (límite de 3 intentos) que dependen solo de la UI y no de invariantes de estado reales. 4) Estados concurrentes entre el escaneo (ScannerSystem) y el inicio de la verificación (VerificationManager).

Objetivo de la sesión de testing	Validar la robustez del flujo completo de verificación de puertas (escaneo → interacción → verificación → resultado) y detectar defectos de null-safety, condiciones de carrera y reglas de negocio antes de la siguiente build.
Condiciones o escenarios especiales que se intentarán	Escanear e interactuar (E) casi simultáneamente; entrar a Play/salir de Play y ejecutar tests inmediatamente después; configurar niveles con más tipos de puerta que spawn points disponibles; agotar los 3 intentos y reingresar a la verificación; dejar de mirar un interactable justo después de recibir un mensaje de FeedbackUI.

Nota: *Puede realizarse un Smoke Check breve al inicio para comprobar que la build inicia, permite acceder al menú y ejecutar las funciones esenciales. El objetivo principal de este laboratorio es la investigación, documentación y gestión de defectos.*

FASE 3 — Preparar el formato de Bug Report

Antes de comenzar la búsqueda de defectos, prepare el formato que se utilizará para cada hallazgo. Cada Bug Report deberá contener, como mínimo, los siguientes campos:

Campo	Información requerida
Bug ID	Identificador único. Ej.: PROYECTO-BUG-001.
Título	Qué ocurre, dónde ocurre y bajo qué condición.
Build / entorno	Versión, plataforma, sistema operativo, hardware o input relevante.
Precondiciones	Estado necesario antes de ejecutar los pasos.
Pasos para reproducir	Secuencia clara y, cuando sea posible, mínima.
Expected Result	Comportamiento que debería ocurrir.
Actual Result	Comportamiento observado realmente.
Repro Rate	Intentos reproducidos / intentos realizados. Ej.: 4/5, 1/10.
Severidad	Crítica / Alta / Media / Baja.
Prioridad	Alta / Media / Baja, según el contexto del proyecto.
Estado	New; o Duplicate / Cannot Reproduce si corresponde.
Evidencia	Screenshot, video, log, consola, configuración, savegame u otra evidencia útil.

Ejemplo de título correcto: “[Gameplay][Build 0.6.12] El jugador atraviesa el suelo al realizar un dash cerca de una pared”.

FASE 4 — Ejecutar el testing del producto

Ejecute la build definida en la estrategia y realice una sesión de testing dirigida a los riesgos seleccionados. Además del flujo normal, explore límites, secuencias inesperadas, acciones simultáneas, cambios rápidos de estado, interrupciones, repetición de acciones y combinaciones que puedan revelar defectos.

- Ejemplos: saltar y atacar simultáneamente; pausar durante una animación; usar una habilidad repetidamente; morir durante una transición; abrir un menú mientras ocurre otra acción; salir y volver a entrar a una zona; repetir una interacción varias veces.

Durante la sesión deberán identificarse y documentarse más de cinco defectos. La entrega mínima será de seis (6) Bug Reports completos y cada uno deberá contar con evidencia.

Nota: *No se considerarán válidos reportes vagos como “el juego falla”, “el personaje se buguea” o “no funciona”. Otra persona debe poder comprender e intentar reproducir el problema a partir del reporte.*

FASE 5 — Investigar al menos un bug mediante Minimal Reproduction

Seleccione al menos uno de los defectos encontrados e intente eliminar pasos innecesarios hasta encontrar la secuencia mínima que provoca el fallo. Registre la secuencia inicial y la secuencia mínima resultante.

Secuencia inicial	Secuencia mínima obtenida
1) Play en modo exploración. 2) Apuntar a una puerta y mantener F (inicia ScanRoutine, 3s). 3) Explorar distintas combinaciones de movimientos, cámara e interacciones mientras el escaneo está activo. 4) Con F aún presionado, presionar E en distintos momentos del escaneo. 5) Observar el comportamiento de la tablet, del movimiento del jugador y del estado de verificación.	1) Escena mínima: 1 jugador (PlayerController + PlayerInteractor + PlayerLook), 1 DoorOption a 2 m, 1 VerificationConsole a 2 m, ScannerSystem y VerificationManager con referencias asignadas. 2) Apuntar a la puerta y mantener F (sin soltar). 3) Esperar ~0.5 s (dentro de la ventana de 3 s de ScanRoutine). 4) Presionar E (sin soltar F). 5) Verificar en el mismo frame: verificationActive == true (VerificationManager.cs:65) y isScanning == true (ScannerSystem.cs:58).

FASE 6 — Elaborar el registro general de bugs

Consolide los hallazgos en una tabla resumen. Los Bug Reports completos deberán presentarse además de este registro.

ID	Área	Resumen	Repro	Severidad	Prioridad	Estado
BUG-001	Controles / Escaneo	Crash (NullReferenceException) si ScannerSystem no está presente/activo en la escena al escanear.	100	Alta	Alta	Nueva
BUG-002	UI / HUD	MissingReferenceException en cascada cuando FeedbackUI.Instance está destruido o ausente; ~12 call-sites sin guard.	100	Alta	Alta	Nueva
BUG-003	Gameplay / Estados concurrentes	Escanear (F) e interactuar (E) casi al mismo tiempo deja el escaneo y la verificación activos en simultáneo.	70	Alta	Alta	Nueva
BUG-004	Gameplay / Progresión de niveles	Si la puerta correcta cae fuera del rango de spawn points, el nivel queda imposible de completar (nivel infalible).	100	Alta	Alta	Nueva
BUG-005	UI / HUD	El prompt de interacción de FeedbackUI queda "pegado" en pantalla al dejar de mirar el interactable.	100	Alta	Alta	Nueva
BUG-006	Gameplay / Reglas de negocio	El límite de 3 intentos es evadible: reingresar a la verificación resetea integridad pero no los strikes.	100	Alta	Alta	Nueva

FASE 7 — Realizar Bug Triage y determinar qué debe resolverse primero

Analice los defectos encontrados. La prioridad no debe asignarse únicamente por severidad. Considere impacto en el jugador, frecuencia, cantidad de usuarios afectados, importancia de la funcionalidad, bloqueo de progreso, existencia de workaround, riesgo para la siguiente build o entrega y dependencias con otras tareas.

Seleccione los tres bugs que deben resolverse primero y justifique la decisión.

Orden	Bug ID	Prioridad	Justificación
1	BUG-004	Alta	Un jugador puede quedar atrapado en un nivel que nunca puede completar; detiene el progreso del núcleo del juego de forma invisible, sin ningún error visible. Impacto directo en retención y experiencia. Fix: en SpawnLevel, si correctIndex >= spawnPoints.Length, redirigir la puerta correcta (p. ej. correctIndex % spawnPoints.Length) y emitir Debug.LogWarning de configuración.
2	BUG-006	Alta	Evade la regla central del minijuego (límite de 3 intentos): el reingreso restaura integridad y avance sin validar HasReachedMaxStrikes(), rompiendo la penalización y el contrato de dificultad. Fix: validar en StartVerification() y no reactivar la consola en FailVerification() al llegar al límite.
3	BUG-002	Media	Crash transversal (MissingReferenceException) en el ciclo Play -> tests con ~12 call-sites sin guard; ya reproducido en vivo. Se prioriza después de los dos anteriores porque no bloquea el progreso permanentemente ni corrompe una regla de negocio. Fix: FeedbackUI.TryShowMessage con guard != null.

FASE 8 — Actualizar Trello con los defectos priorizados

Incorpore al tablero de Trello los tres bugs de mayor prioridad como nuevas tarjetas o ítems del backlog. Cada tarjeta debe contener como mínimo: Bug ID, título, responsable, prioridad, descripción y evidencia o vínculo al reporte.

Adjunte en el informe una captura de Trello donde se observen los defectos incorporados a la planificación del proyecto.

VIII. Entrega de resultados

Cada equipo entregará un único documento de laboratorio que incluya:

- Identificación del proyecto y de los integrantes del equipo.
- Captura del tablero Trello inicial.
- Listado de las cinco funcionalidades finalizadas seleccionadas para el testing.
- Estrategia de testing: alcance, build, entorno, riesgos y objetivo.
- Registro resumen de mínimo seis bugs.
- Seis Bug Reports completos con evidencia.
- Minimal Reproduction de al menos un bug.
- Resultado del Bug Triage y justificación de los tres defectos que deben resolverse primero.
- Enlace al Trello - https://trello.com/invite/b/6abdc46e2602c02661d2709c/ATTI0cb9f622130c31c5e6f1a34b4c7b85d87B9DE308/testing-tesis-tecsupverse
- Conclusiones del equipo.

NOTE: EN CASO DE LOS REPORTE EN SU FASE CORRESPONDIENTE ESCRIBA LA URL DE SU DRIVE DONDE SE DEBE DE ENCONTRAR LOS REPORTES Y EVIDENCIAS

Conclusiones:

Responder de forma breve y sustentada:

1. **¿Cuál fue el defecto más crítico encontrado y por qué?**

   B BUG-004 (nivel infalible cuando la puerta correcta supera el número de spawn points). Es el más crítico porque bloquea el progreso del núcleo del juego de forma completamente invisible: no genera ningún error ni excepción, el jugador simplemente no puede ganar, lo que impacta de manera directa en la retención y en la percepción de justicia del minijuego.

2. **¿Cuál fue el bug más difícil de reproducir?**

BUG-003 (concurrencia entre escaneo y verificación), porque depende de una ventana de tiempo muy precisa entre dos máquinas de estado asíncronas (ScanRoutine y StartVerification) que no se comunican entre sí; su tasa de reproducción fue de aproximadamente 70% y requirió aislar los pasos mínimos (ver Fase 5) para diagnosticarlo con confianza.

3. **¿Qué funcionalidad presentó mayor riesgo durante las pruebas?**

El flujo de verificación de puertas en su conjunto (VerificationManager, ScannerSystem, DoorSpawner y FeedbackUI), por depender de singletons estáticos accedidos sin validación de nulidad y de referencias entre managers sin guards; una escena mal configurada transforma fácilmente un problema de contenido en un crash o en un estado de juego roto.

4. **¿La prioridad asignada coincidió siempre con la severidad? Explique un caso.**

No siempre. BUG-002 tiene severidad Alta (provoca crashes/excepciones y ya se reprodujo en vivo) pero se priorizó como P1 y no P0, porque, a diferencia de BUG-004 y BUG-006, no bloquea el progreso del jugador de forma permanente ni compromete una regla de negocio del juego. En sentido inverso, BUG-005 es 100% reproducible pero tiene severidad Baja y prioridad Baja, porque es solo un problema visual sin impacto funcional: la frecuencia de reproducción no determinó por sí sola la prioridad.

5. **¿Qué cambio debería realizar el equipo en su proceso de desarrollo para evitar que alguno de los defectos vuelva a aparecer?**

TDD forzado para reglas de estado (ya iniciado con las 14 pruebas EditMode): todo método con validación de negocio (SetMaxStrikes, StartVerification) debe nacer con su prueba. 2) Null-safety sistemático: un helper centralizado (p. ej. TryShowMessage) y prohibir llamadas a Instance.X() sin guard en code review. 3) Validación de configuración en el editor: usar OnValidate() o advertencias en SpawnLevel/StartVerification cuando los managers detecten referencias o arreglos incompletos. 4) Checklist de QA de contenido: cada LevelConfig debe validarse contra sus spawnPoints antes de marcarse como "listo".

Flujo de trabajo del laboratorio:

TRELLO → ALCANCE → ESTRATEGIA → TESTING → ≥ 6 BUGS + EVIDENCIA → INVESTIGACIÓN → TRIAGE → TRELLO

LinkDrive:

https://drive.google.com/file/d/1Gok9Bl8gKjRLK2JJmlBChkMrQcKUOJGw/view?usp=sharing